SENATI

ESCUELA DE TECNOLOGÍAS DE LA INFORMACIÓN

TALLER PRÁCTICO

APLICACIONES WEB PROGRESIVAS

TEMA: IMPLEMENTAR UNA APP MANIFEST

Caso práctico: “Backus Ventas PWA”

1. Datos generales

Elemento	Descripción
Programa	Tecnologías de la Información
Unidad	Aplicaciones Web Progresivas
Tema	Implementar una App Manifest
Duración	180 minutos
Nivel	Básico
Tecnología	Angular 19 + TypeScript
PWA	Angular Service Worker
Persistencia	IndexedDB
Producto final	PWA “Backus Ventas”
Evidencia	Proyecto + capturas de funcionamiento
Nota académica “Backus Ventas” es un caso educativo. Los productos, precios, datos, API y procesos utilizados son ficticios y se emplean únicamente con fines de aprendizaje.	

2. Objetivo del taller

Objetivo general

Desarrollar una aplicación App Manifest Angular en un tiempo establecido y bajo condiciones reales de trabajo.

Al finalizar el taller, el estudiante será capaz de:

- Crear un proyecto Angular con soporte PWA.
- Configurar el App Manifest.
- Personalizar nombre, iconos, colores y modo de visualización.
- Comprender la relación entre App Manifest y Service Worker.
- Personalizar el comportamiento de determinadas solicitudes según escenarios online/offline.
- Configurar Runtime Caching básico.
- Implementar persistencia básica de datos mediante IndexedDB.
- Probar la aplicación sin conexión.
- Obtener evidencias técnicas del funcionamiento.

3. Conocimientos tecnológicos

1. Personalizar las respuestas a las solicitudes de red dependiendo del estado en línea/fuera de línea.

- Service Worker
- Cache Storage
- ngsw-config.json
- Estrategias performance y freshness
- Detección del estado de conexión
- Pruebas offline

2. Persistencia de datos y mensajes.

- IndexedDB
- Almacenamiento de pedidos
- Almacenamiento de mensajes
- Recuperación de información local

4. Conocimiento práctico

Configura el manifiesto de una aplicación web.

- Nombre de aplicación
- Nombre corto
- Iconos
- Color de tema
- Color de fondo
- Página inicial
- Modo standalone

5. Situación problemática

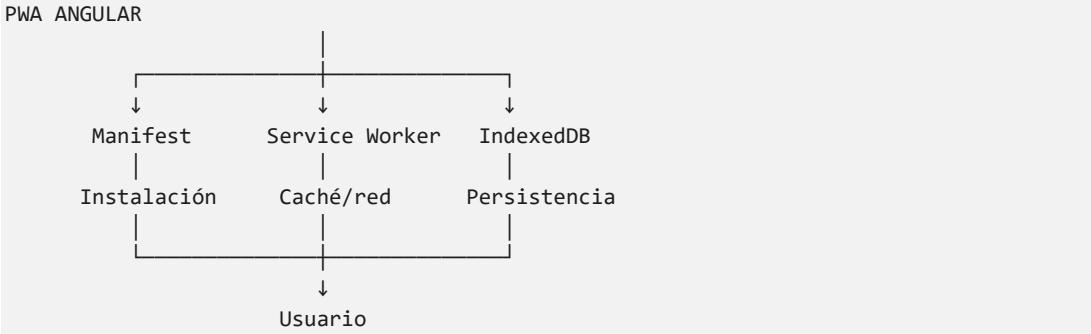
Una empresa necesita una aplicación móvil para que sus vendedores puedan consultar información comercial y registrar pedidos. Los vendedores trabajan en diferentes lugares y pueden encontrarse con señal débil, conexión intermitente, pérdida temporal de Internet o ausencia de conexión.

Reto
¿Cómo podemos convertir una aplicación Angular tradicional en una PWA básica preparada para escenarios online y offline?

Pregunta orientadora

¿Cómo podemos construir una aplicación web que pueda instalarse en un dispositivo, conserve algunos recursos disponibles, responda de manera diferente cuando existe o no existe conexión y permita conservar información localmente?

6. Arquitectura conceptual del taller



Manifest: define principalmente cómo se identifica y presenta la aplicación instalada.

Service Worker: permite gestionar determinadas solicitudes, caché y comportamiento offline.

IndexedDB: permite almacenar datos estructurados de forma persistente en el navegador.

7. Distribución de los 180 minutos

Fase	Actividad	Tiempo
1	Saberes previos y situación problemática	15 min
2	Crear proyecto Angular + PWA	20 min
3	Configurar App Manifest	25 min
4	Personalizar respuestas online/offline	30 min
5	Implementar persistencia con IndexedDB	40 min
6	Pruebas técnicas y validación	20 min
7	Evidencias y reflexión	15 min
8	Evaluación y cierre	15 min
TOTAL		180 min

FASE 1 – SABERES PREVIOS

Tiempo: 15 minutos

Antes de programar, responde:

1. ¿Qué es una PWA?

2. ¿Qué diferencia existe entre una aplicación web y una PWA?

3. ¿Qué es un App Manifest?

4. ¿Qué función cumple un Service Worker?

5. ¿Qué significa trabajar offline?

6. ¿Qué información debería mantenerse disponible sin Internet?

7. ¿Qué es persistencia de datos?

8. ¿Qué tecnologías del navegador permiten almacenar información?

Actividad rápida
Un vendedor está registrando un pedido y pierde la conexión justo antes de enviarlo. ¿Debería perderse el pedido? ¿Qué solución propondrías?

FASE 2 – CREAR EL PROYECTO ANGULAR PWA

Tiempo: 20 minutos

Paso 1. Verificar herramientas

```
node -v
npm -v
ng version
```

El estudiante debe visualizar las versiones instaladas.

```
PS C:\angular1> node -v
v20.11.1
PS C:\angular1> npm -v
10.5.0
PS C:\angular1> ng version
```

```
PS C:\angular1> ng version

     _                      _                 ____ _     ___
    / \   _ __   __ _ _   _| | __ _ _ __     / ___| |   |_ _|
   / △ \ | '_ \ / _` | | | | |/ _` | '__|   | |   | |    | |
  / ___ \| | | | (_| | |_| | | (_| | |      | |___| |___ | |
 /_/   \_\_| |_|\__, |\__,_|_|\__,_|_|       \____|_____|___|
                |___/


Angular CLI: 17.3.2
Node: 20.11.1
Package Manager: npm 10.5.0
OS: win32 x64

Angular:
...

Package                         Version
---------------------------------------------------------
@angular-devkit/architect       0.1703.2 (cli-only)
@angular-devkit/core            17.3.2 (cli-only)
@angular-devkit/schematics      17.3.2 (cli-only)
@schematics/angular             17.3.2 (cli-only)

PS C:\angular1>
```

Paso 2. Crear el proyecto

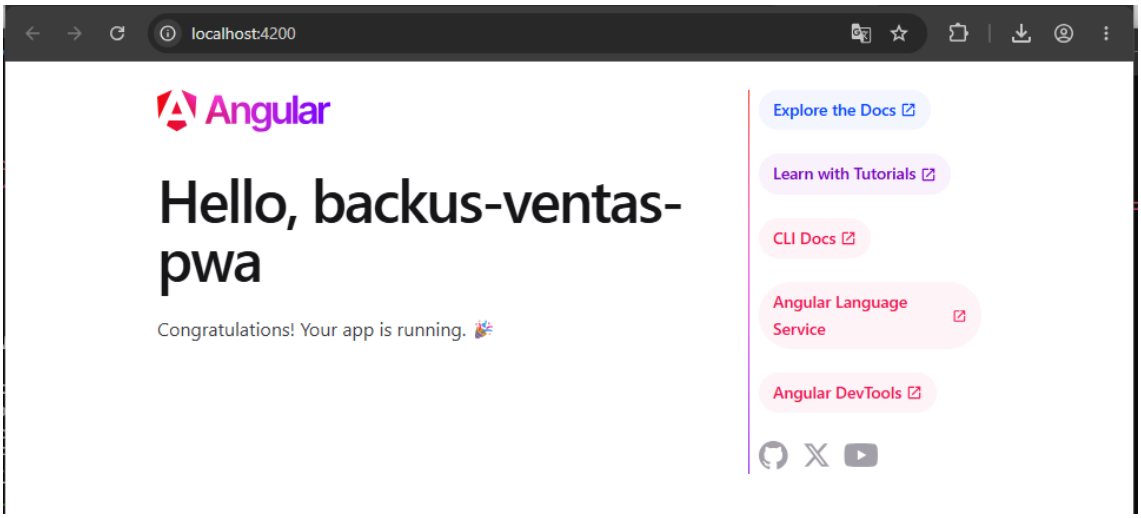
```
ng new backus-ventas-pwa --routing --style=scss
cd backus-ventas-pwa
```

Paso 3. Ejecutar la aplicación

```
ng serve
```

Abrir: http://localhost:4200

Verificación
El estudiante debe observar la aplicación Angular funcionando.



FASE 3 – AGREGAR SOPORTE PWA

Ejecutar:

```
ng add @angular/pwa
```

Revisar los archivos y elementos generados, especialmente el manifest, la configuración del Service Worker y los iconos.

manifest.webmanifest

src > { } manifest.webmanifest > ...

```
{
  "name": "backus-ventas-pwa",
  "short_name": "backus-ventas-pwa",
  "theme_color": "#1976d2",
  "background_color": "#fafafa",
  "display": "standalone",
  "scope": "./",
  "start_url": "./",
  "icons": [
    {
      "src": "assets/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "assets/icons/icon-96x96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "maskable any"
    },
```

FASE 4 – CONFIGURAR EL APP MANIFEST

Tiempo: 25 minutos

Paso 1. Abrir el Manifest

Localizar el archivo manifest.webmanifest. La ubicación puede variar ligeramente según la configuración del proyecto Angular; utiliza el archivo de manifest generado por ng add @angular/pwa.

Paso 2. Personalizar la aplicación

```
"name": "Backus Ventas PWA",
"short_name": "Backus",
"start_url": "/",
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#0033a0"
```

Paso 3. Revisar los iconos

```
"icons": [
  {
    "src": "icons/icon-192x192.png",
    "sizes": "192x192",
    "type": "image/png"
  },
  {
    "src": "icons/icon-512x512.png",
    "sizes": "512x512",
    "type": "image/png"
  }
]
```

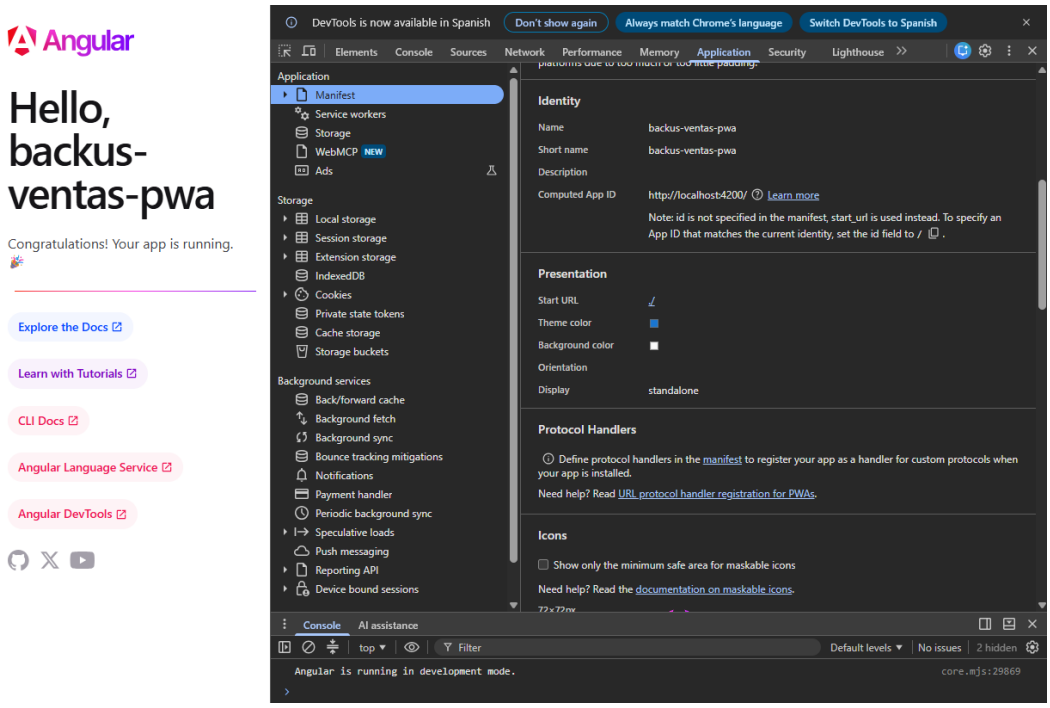
Pregunta: ¿Por qué una PWA necesita diferentes tamaños de iconos?

Respuesta esperada: porque los dispositivos y contextos de instalación pueden requerir diferentes dimensiones de iconos.

Paso 4. Verificar el Manifest

Abrir DevTools (F12) → Application → Manifest. Revisar nombre, iconos, colores y datos de instalación.

Evidencia 01
Captura donde se observe el nombre de la aplicación, iconos y propiedades del Manifest.



FASE 5 – CREAR EL CATÁLOGO

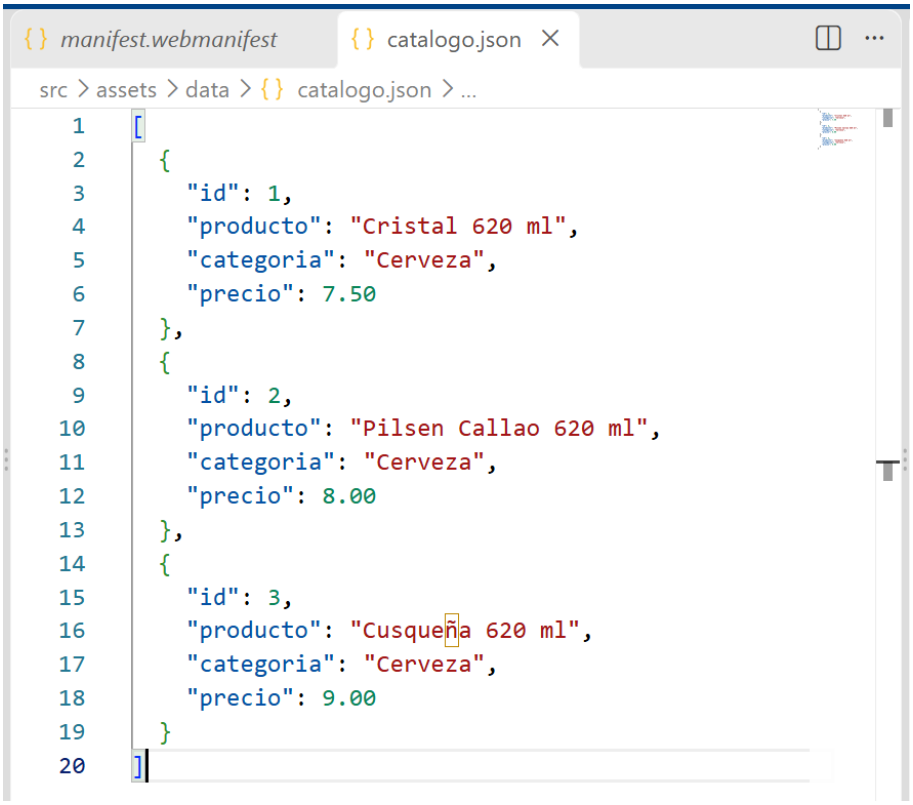
Tiempo: 30 minutos

Crear el archivo:

public/data/catalogo.json

```
[
  {
    "id": 1,
    "producto": "Cristal 620 ml",
    "categoria": "Cerveza",
    "precio": 7.50
  },
  {
    "id": 2,
    "producto": "Pilsen Callao 620 ml",
    "categoria": "Cerveza",
    "precio": 8.00
  },
  {
    "id": 3,
    "producto": "Cusqueña 620 ml",
    "categoria": "Cerveza",
    "precio": 9.00
  }
]
```

Nota
Si no existe la carpeta public, deberá emplear la carpeta assets
Los productos y precios son completamente ficticios y solo tienen finalidad educativa.



Paso 6. Configurar HttpClient

Revisar src/app/app.config.ts. Debe existir una configuración equivalente a:

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient()
  ]
};
```

catalogo.json | app.config.ts

src > app > app.config.ts

```ts
import { ApplicationConfig, isDevMode } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';
import { provideServiceWorker } from '@angular/service-worker';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes), provideServiceWorker('ngsw-worker.js', {
      enabled: !isDevMode(),
      registrationStrategy: 'registerWhenStable:30000'
    })]
};

```

Paso 7. Crear el servicio

ng generate service services/catalogo

```
PS C:\angular1\backus-ventas-pwa> ng generate service services/catalogo
CREATE src/app/services/catalogo.service.spec.ts (383 bytes)
CREATE src/app/services/catalogo.service.ts (146 bytes)
PS C:\angular1\backus-ventas-pwa>
```

```
src
└── app
    └── services
        ├── catalogo.service.ts
        └── catalogo.service.spec.ts
```

catalogo.json | catalogo.service.ts

src > app > services > catalogo.service.ts

```ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CatalogoService {

  constructor() { }
}

```

Implementar el servicio:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
export interface Producto {
  id: number;
  producto: string;
  categoria: string;
  precio: number;
}
@Injectable({
  providedIn: 'root'
})
export class CatalogoService {
  private url = '/assets/data/catalogo.json';
  constructor(private http: HttpClient) {}
  obtenerCatalogo(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.url);
  }
}
```

src > app > services > catalogo.service.ts

```typescript
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
export interface Producto {
  id: number;
  producto: string;
  categoria: string;
  precio: number;
}
@Injectable({
  providedIn: 'root'
})
export class CatalogoService {
  private url = '/assets/data/catalogo.json';
  constructor(private http: HttpClient) {}
  obtenerCatalogo(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.url);
  }
}

```

Paso 8. Configurar Runtime Caching

Abrir ngsw-config.json y agregar un dataGroup para el catálogo:

```
{
  "dataGroups": [
    {
      "name": "catalogo",
      "urls": [
        "/data/catalogo.json"
      ],
      "cacheConfig": {
        "strategy": "performance",
        "maxSize": 20,
        "maxAge": "1d"
      }
    }
  ]
}
```

```
  ],
  "dataGroups": [
    {
      "name": "catalogo",
      "urls": [
        "/data/catalogo.json"
      ],
      "cacheConfig": {
        "strategy": "performance",
        "maxSize": 20,
        "maxAge": "1d"
      }
    }
  ]
}
```

Interpretación:

- performance: prioriza la respuesta disponible en caché.
- maxSize: limita el número de respuestas almacenadas.
- maxAge: define el tiempo máximo de permanencia.

Decisión técnica
El catálogo académico puede tolerar cierta antigüedad, por lo que se puede priorizar disponibilidad y rapidez con strategy: performance.

FASE 6 – DETECTAR ONLINE / OFFLINE

Tiempo: parte de los 30 minutos

En el componente principal podemos consultar:

```
navigator.onLine
```

Ejemplo conceptual:

```
online = navigator.onLine;

window.addEventListener('online', () => {
  this.online = true;
});

window.addEventListener('offline', () => {
  this.online = false;
});
```

La interfaz puede mostrar: CONECTADO o SIN CONEXIÓN.

Importante
navigator.onLine es útil para conocer el estado de conectividad del navegador, pero no garantiza que

nuestro servidor o API esté disponible. Una aplicación profesional debe considerar errores, timeout, disponibilidad de API y recuperación desde caché.

catalogo.json | catalogo.service.ts | ngsw-config.json | app.component.ts

src > app > app.component.ts

```ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.scss'
})
export class AppComponent {
  title = 'backus-ventas-pwa';
}

```

catalogo.json | catalogo.service.ts | ngsw-config.json | app.component.ts

src > app > app.component.ts

```ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.scss'
})
export class AppComponent {

  title = 'backus-ventas-pwa';

  online = navigator.onLine;

  constructor() {

    window.addEventListener('online', () => {
      this.online = true;
    });

    window.addEventListener('offline', () => {
      this.online = false;
    });

  }

}
```

FASE 7 – PERSISTENCIA CON INDEXEDDB

Tiempo: 40 minutos

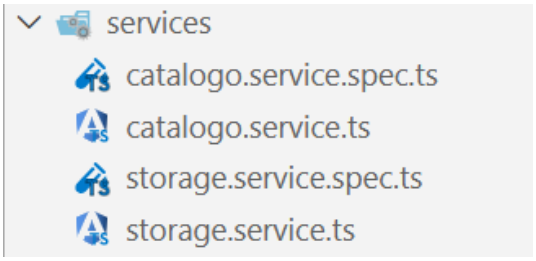
Problema: ¿Qué ocurre si el usuario registra un mensaje o pedido mientras está offline? Necesitamos almacenarlo localmente.

Ejemplos: pedido, mensaje, configuración, formulario o información temporal.

Paso 11. Crear servicio de persistencia

```
ng generate service services/storage
```

```
PS C:\angular1\backus-ventas-pwa> ng generate service services/storage
CREATE src/app/services/storage.service.spec.ts (378 bytes)
CREATE src/app/services/storage.service.ts (145 bytes)
PS C:\angular1\backus-ventas-pwa>
```



src > app > services > storage.service.ts

```
1   import { Injectable } from '@angular/core';
2
3   @Injectable({
4     providedIn: 'root'
5   })
6   export class StorageService {
7
8     constructor() { }
9   }
10
```

Implementar una versión básica:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class StorageService {

  private dbName = 'backusVentasDB';
  private storeName = 'pedidos';
```

```
async abrirDB(): Promise<IDBDatabase> {

  return new Promise((resolve, reject) => {

    const request = indexedDB.open(this.dbName, 1);

    request.onupgradeneeded = () => {

      const db = request.result;

      if (!db.objectStoreNames.contains(this.storeName)) {
        db.createObjectStore(this.storeName, {
          keyPath: 'id',
          autoIncrement: true
        });
      }
    };

    request.onsuccess = () => {
      resolve(request.result);
    };

    request.onerror = () => {
      reject(request.error);
    };

  });
}

async guardarPedido(pedido: any): Promise<void> {

  const db = await this.abrirDB();

  return new Promise((resolve, reject) => {

    const transaction =
      db.transaction(this.storeName, 'readwrite');

    const store =
      transaction.objectStore(this.storeName);

    store.add({
      ...pedido,
      fecha: new Date().toISOString(),
      estado: 'pendiente'
    });

    transaction.oncomplete = () => {
      resolve();
    };

    transaction.onerror = () => {
      reject(transaction.error);
    };
```

```
    });
  }
}
```

src > app > services > storage.service.ts

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class StorageService {

  private dbName = 'backusVentasDB';
  private storeName = 'pedidos';

  async abrirDB(): Promise<IDBDatabase> {

    return new Promise((resolve, reject) => {

      const request = indexedDB.open(this.dbName, 1);

      request.onupgradeneeded = () => {

        const db = request.result;

        if (!db.objectStoreNames.contains(this.storeName)) {
          db.createObjectStore(this.storeName, {
            keyPath: 'id',
            autoIncrement: true
          });
        }
      };

      request.onsuccess = () => {
        resolve(request.result);
```

Paso 12. Comprender la persistencia

La aplicación crea una base local llamada backusVentasDB y un almacén llamado pedidos. Cada registro puede contener id, producto, cantidad, fecha y estado.

```
{
  "producto": "Cristal 620 ml",
  "cantidad": 2,
  "estado": "pendiente"
}
```

Paso 13. Flujo offline

```
Usuario
   ↓
Registra pedido
   ↓
¿Existe conexión?
   │
   ├── SÍ → API/Servidor
   │
   └── NO → IndexedDB
              ↓
         Estado: pendiente
```

Paso 14. Cuando regresa Internet

```
CONEXIÓN RECUPERADA
        ↓
Consultar pedidos pendientes
        ↓
Enviar a API
        ↓
Servidor responde
        ↓
Actualizar estado
        ↓
Confirmar al usuario
```

Alcance del taller
En este taller básico se implementa la persistencia local. La sincronización completa con una API real será una ampliación posterior. Runtime Caching no convierte automáticamente los pedidos POST en una cola offline.

FASE 8 – IMPLEMENTAR UN MENSAJE OFFLINE

Ejercicio práctico

```
---------------------------------
      MENSAJE DEL VENDEDOR
---------------------------------

Mensaje:
[_____________________________]

[ GUARDAR ]

Estado:
🟢 Conectado
o
🔴 Sin conexión
---------------------------------
```

Al presionar Guardar:

- • Si existe conexión: mostrar mensaje de procesamiento.
- • Si no existe conexión: almacenar el mensaje localmente.

html

```
<div class="contenedor">


  <h1>Backus Ventas PWA</h1>


  @if (online) {
    <div class="estado conectado">
      🟢 CONECTADO
    </div>
  } @else {
    <div class="estado desconectado">
      🔴 SIN CONEXIÓN
    </div>
  }
```

```
</div>
```

Scss

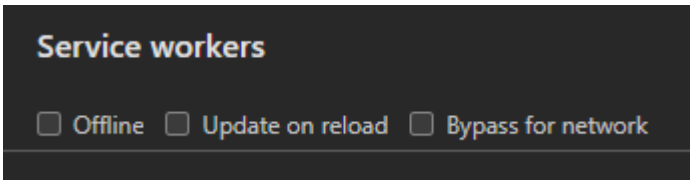
```
.contenedor {
  max-width: 800px;
  margin: 40px auto;
  padding: 20px;
  font-family: Arial, sans-serif;
}

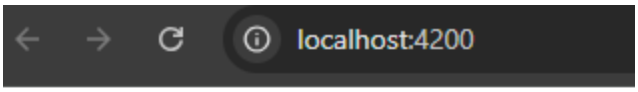
h1 {
  text-align: center;
}

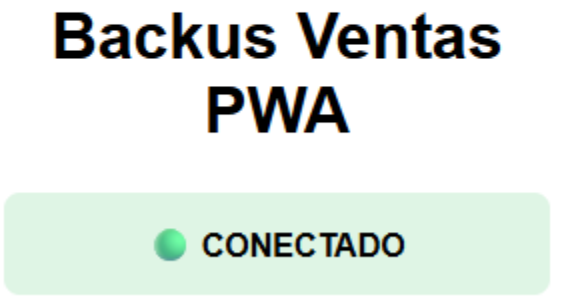
.estado {
  padding: 15px;
  margin: 20px 0;
  border-radius: 8px;
  text-align: center;
  font-weight: bold;
}

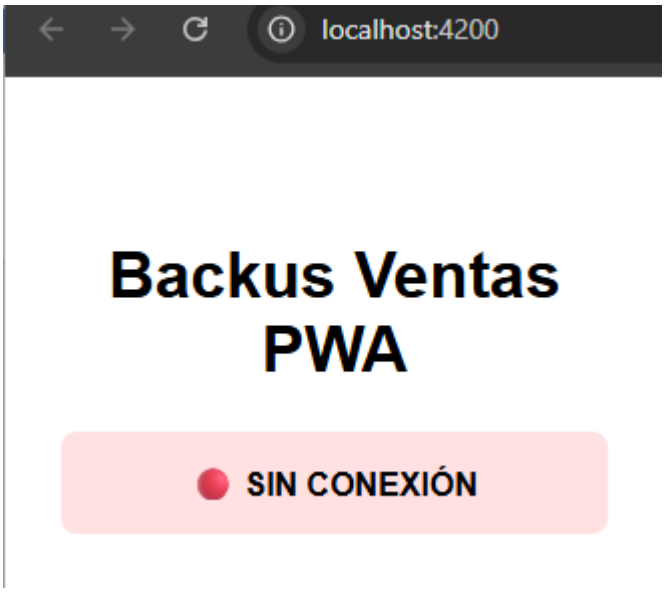
.conectado {
  background: #dff5e5;
}

.desconectado {
  background: #ffe1e1;
}
```





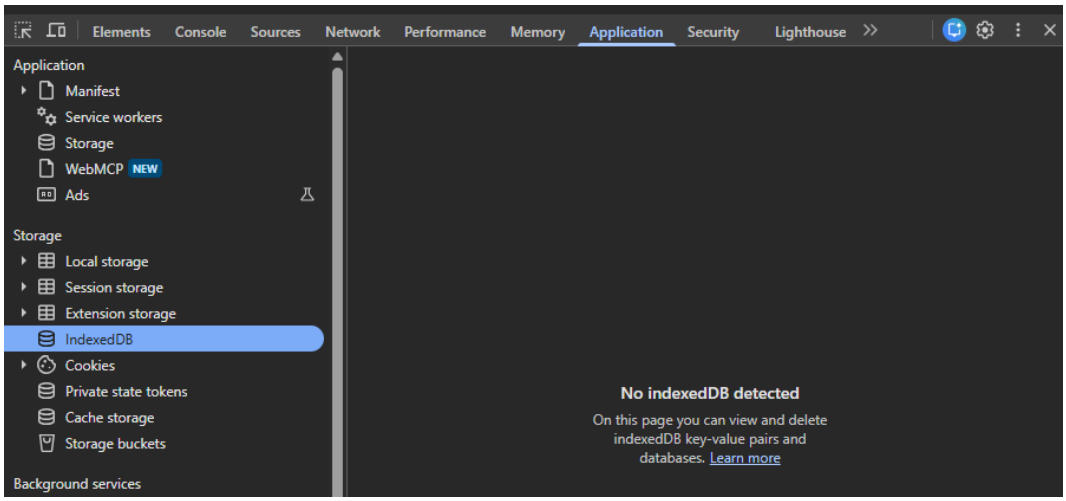




Paso 15. Prueba de persistencia

Abrir F12 → Application → IndexedDB → backusVentasDB → pedidos.

Evidencia 02
Capturar IndexedDB mostrando la base de datos, el almacén pedidos y el registro guardado.



FASE 9 – PRUEBA DEL SERVICE WORKER

Tiempo: 20 minutos

Para probar correctamente el Service Worker, realizar una compilación de producción:

```
ng build --configuration production
```

La salida normalmente estará en una estructura semejante a:

```
dist/
└── backus-ventas-pwa/
    └── browser/
```

```
PS C:\angular1\backus-ventas-pwa> ng build --configuration production
Initial chunk files   | Names         |  Raw size | Estimated transfer size
main-5SXQKZGN.js      | main          | 178.58 kB |               48.42 kB
polyfills-FFHMD2TL.js | polyfills     |  33.71 kB |               11.02 kB
styles-5INURTSO.css   | styles        |   0 bytes |                0 bytes

                      | Initial total | 212.29 kB |               59.44 kB

Output location: C:\angular1\backus-ventas-pwa\dist\backus-ventas-pwa

Application bundle generation complete. [3.842 seconds]

PS C:\angular1\backus-ventas-pwa>
```

Servir la aplicación:

```
npx http-server ./dist/backus-ventas-pwa/browser -p 8080
```

Abrir: http://localhost:8080

```
PS C:\angular1\backus-ventas-pwa> npx http-server ./dist/backus-ventas-pwa/browser -p 8080
Starting up http-server, serving ./dist/backus-ventas-pwa/browser

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://172.16.20.5:8080
  http://192.168.56.1:8080
  http://192.168.205.1:8080
  http://192.168.32.1:8080
  http://127.0.0.1:8080
Hit CTRL-C to stop the server
```

Recomendación
Para esta práctica se recomienda localhost, porque es un contexto seguro reconocido por el navegador para las pruebas locales de Service Worker.

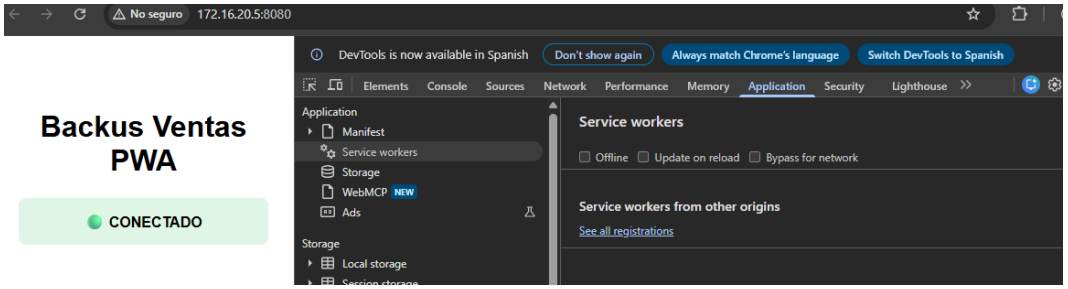




Paso 17. Verificar Service Worker

F12 → Application → Service Workers. Verificar que el Service Worker esté activo.

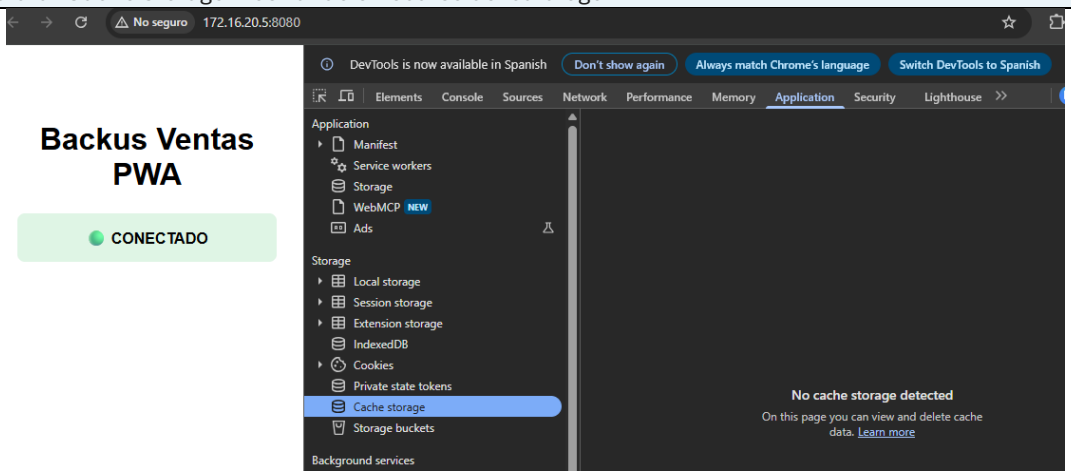
Evidencia 03
Capturar Service Worker mostrando estado activo/Activated.

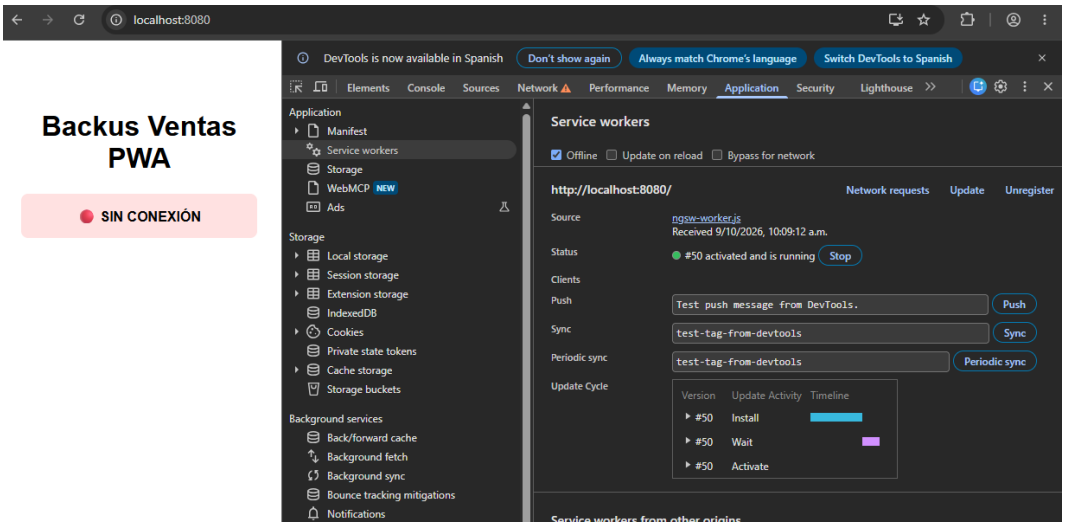


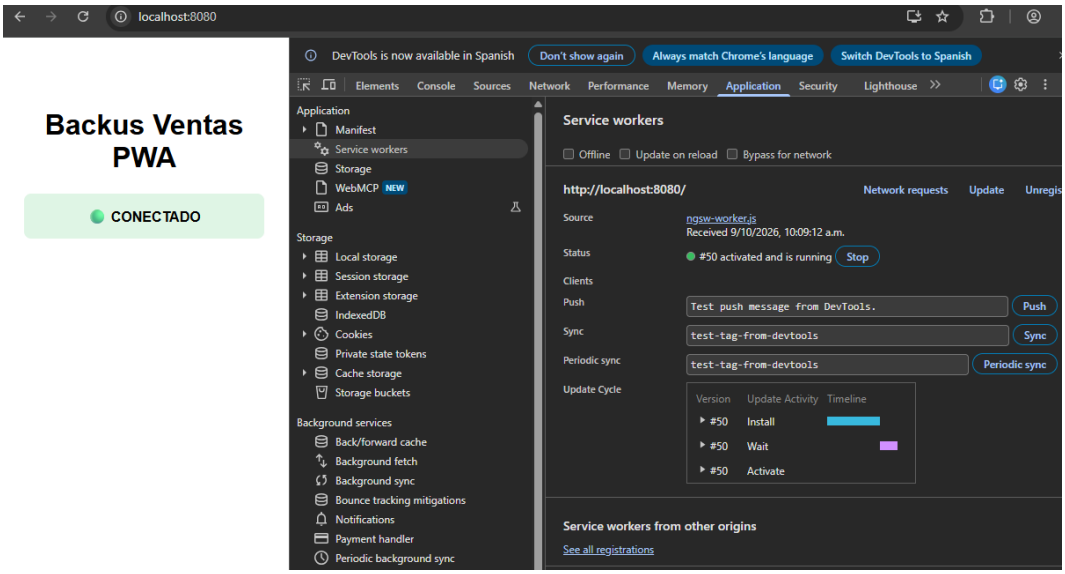
Paso 18. Verificar Cache Storage

F12 → Application → Cache Storage. Buscar los recursos generados por la PWA y verificar el recurso /data/catalogo.json después de haberlo solicitado.

Evidencia 04
Capturar Cache Storage mostrando el recurso del catálogo.







FASE 10 – PRUEBA OFFLINE

Con Internet disponible:

1. Abrir la PWA.
2. Consultar el catálogo.
3. Verificar que los productos aparecen.
4. Revisar Cache Storage.

Luego: DevTools → Network → Offline → actualizar la aplicación.

Observar si el catálogo continúa disponible. Si la configuración y el caché se generaron correctamente, el Service Worker puede responder con la información almacenada.

```
Internet
    ↓
OFFLINE
    ↓
Service Worker
    ↓
Cache Storage
    ↓
Catálogo disponible
```

Evidencia 05
Capturar Network en modo Offline, la aplicación funcionando y el catálogo disponible.

Paso 22. Probar IndexedDB sin conexión

Con el navegador en Offline, registrar un pedido o mensaje y revisar Application → IndexedDB → backusVentasDB → pedidos.

Evidencia 06
Capturar el registro persistido.

FASE 11 – COMPARAR LOS DOS MECANISMOS

Necesidad	Tecnología
Recursos de la aplicación	Cache Storage
Respuestas GET que queremos reutilizar	Cache / Service Worker
Catálogo cacheado	Cache Storage
Pedidos pendientes	IndexedDB
Mensajes pendientes	IndexedDB
Datos estructurados	IndexedDB
Instalación de la aplicación	App Manifest
Gestión de determinadas solicitudes	Service Worker

FASE 12 – ACTIVIDAD DE ANÁLISIS

Caso A

El logotipo de la empresa. ¿Dónde lo almacenarías?

Respuesta esperada: Cache / recursos de la PWA.

Caso B

Catálogo de productos. ¿Dónde podría almacenarse para disponibilidad offline?

Respuesta esperada: Cache Storage mediante Service Worker, según la estrategia definida.

Caso C

Pedido creado sin conexión. ¿Dónde debería persistirse?

Respuesta esperada: IndexedDB.

Caso D

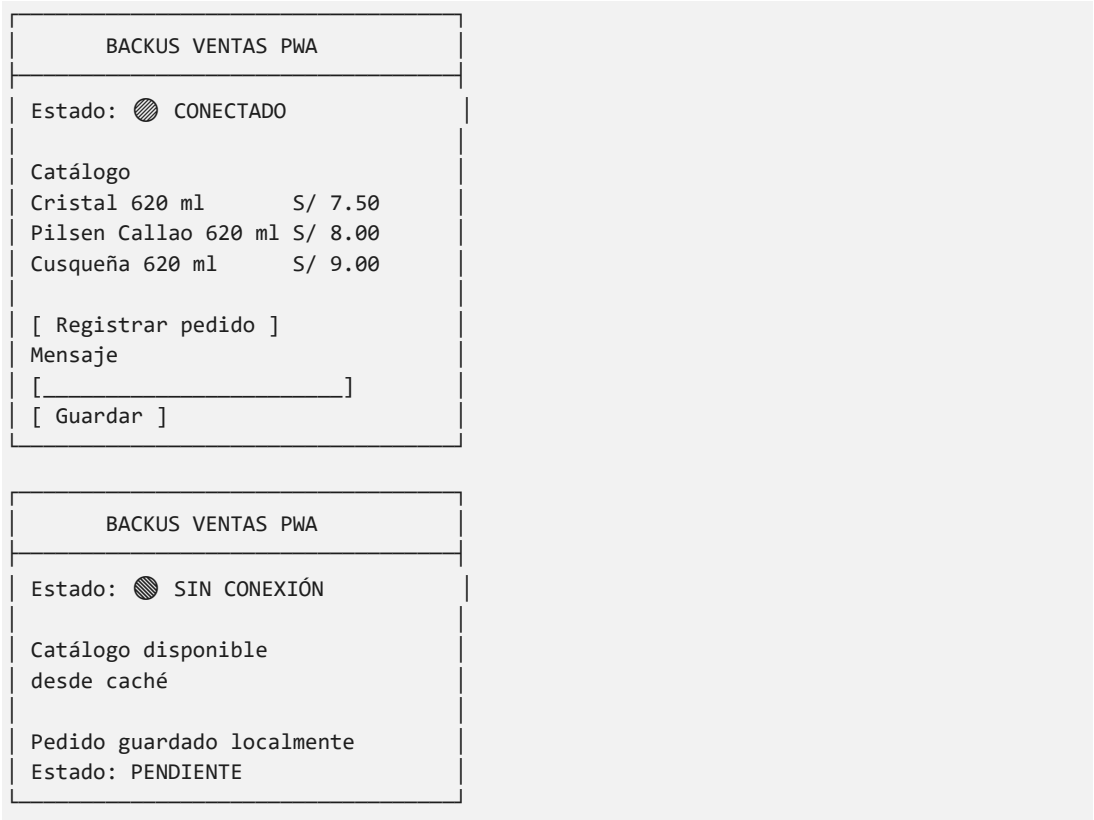
Nombre, iconos y colores de la aplicación. ¿Dónde se configuran?

Respuesta esperada: App Manifest.

FASE 13 – EVIDENCIAS DEL ESTUDIANTE

- Evidencia 1: Manifest configurado.
- Evidencia 2: Aplicación ejecutándose.
- Evidencia 3: Service Worker activo.
- Evidencia 4: Cache Storage.
- Evidencia 5: Aplicación funcionando offline.
- Evidencia 6: IndexedDB con información persistida.
- Evidencia 7: Aplicación instalada como PWA, cuando el navegador/dispositivo lo permita.

Producto final



FASE 14 – REFLEXIÓN POSTINSTRUCCIONAL

Tiempo: 15 minutos

1. ¿Cuál es la función del App Manifest?

2. ¿Qué diferencia existe entre Manifest y Service Worker?

3. ¿Qué problema resuelve el Service Worker?

4. ¿Qué diferencia existe entre Cache Storage e IndexedDB?

5. ¿Por qué un catálogo podría utilizar performance?

6. ¿Por qué determinados datos deberían priorizar la red?

7. ¿Qué ocurre cuando el usuario pierde conexión?

8. ¿Cómo se conserva un pedido offline?

9. ¿Qué evidencia demuestra que el Service Worker funciona?

10. ¿Qué mejorarías en esta PWA para convertirla en una solución empresarial?

Evaluación del taller – Rúbrica de 100 puntos

Criterio	Puntaje
Creación y configuración de la PWA	15
Configuración del App Manifest	20
Configuración del Service Worker / Runtime Caching	20
Funcionamiento online/offline	15
Persistencia mediante IndexedDB	15
Evidencias técnicas	10
Orden, buenas prácticas y presentación	5
TOTAL	100

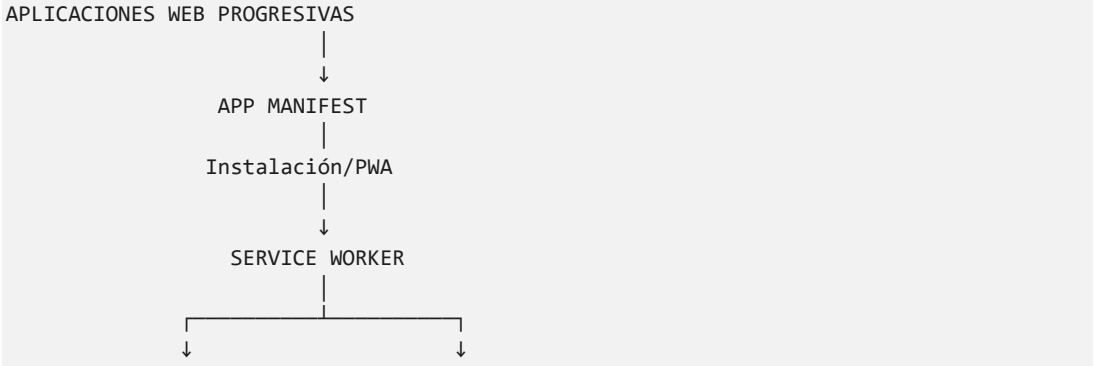
Lista de cotejo

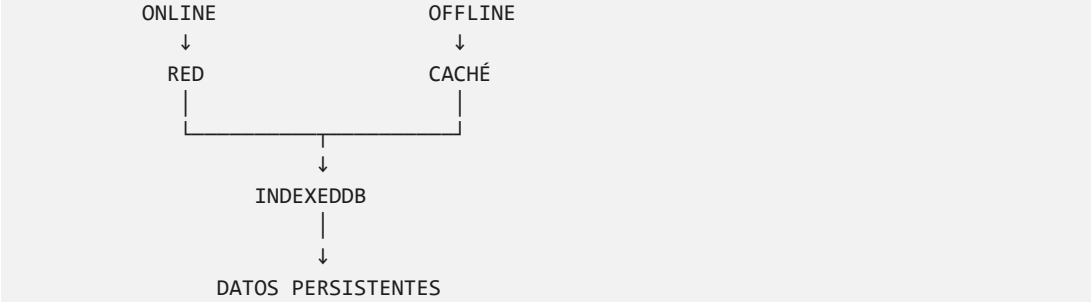
Indicador	Sí	No
Creó correctamente el proyecto Angular	☐	☐
Agregó soporte PWA	☐	☐
Configuró el Manifest	☐	☐
Personalizó nombre e iconos	☐	☐
Configuró display: standalone	☐	☐
Configuró Runtime Caching	☐	☐
Verificó el Service Worker	☐	☐
Probó la aplicación offline	☐	☐
Implementó IndexedDB	☐	☐
Persistió un pedido/mensaje	☐	☐
Presentó evidencias	☐	☐

Distribución final del tiempo

Tiempo	Actividad
00–15 min	Inicio y saberes previos
15–35 min	Creación de PWA
35–60 min	App Manifest
60–90 min	Online / Offline
90–130 min	Persistencia
130–150 min	Pruebas
150–165 min	Prueba offline
165–180 min	Evaluación y reflexión

Conclusión del taller





Idea clave para el estudiante
El App Manifest permite definir la identidad y experiencia de instalación de la aplicación; el Service Worker permite gestionar determinadas solicitudes y recursos ante diferentes condiciones de red; e IndexedDB permite conservar datos estructurados localmente.

Por tanto, una PWA profesional no consiste simplemente en “hacer que una aplicación funcione sin Internet”, sino en diseñar qué recursos deben estar disponibles, qué información necesita actualizarse, qué datos deben persistir y cómo debe responder la aplicación ante cambios de conectividad.